

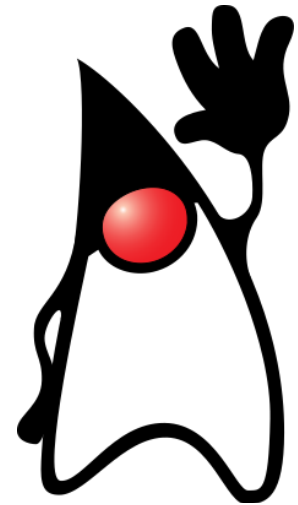
Java SE and JVM Fundamentals

Aleksandra Mileva (aleksandra.mileva@ugd.edu.mk)

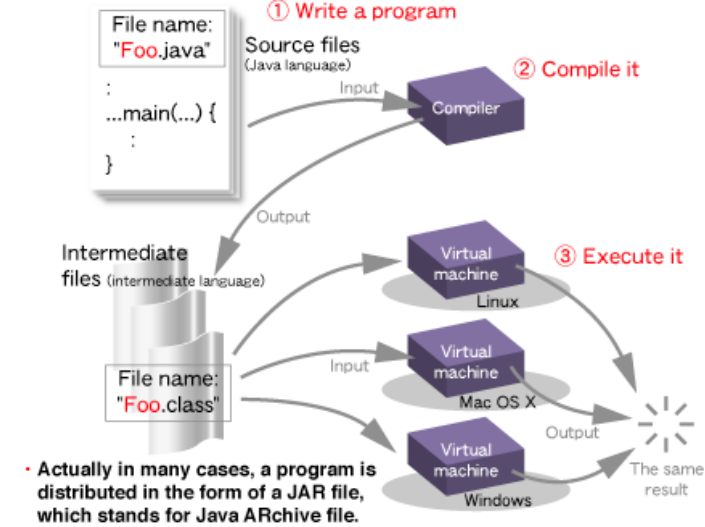
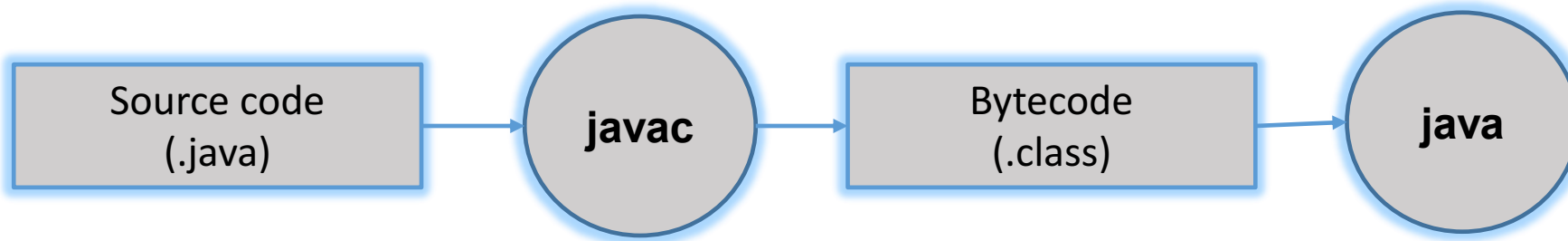
What is Java?



- 1991 – 1995 - it was first developed by **James Gosling** at Sun Microsystems (2010 - Oracle Corporation)
 - Oak, Green Project
- Java is a **general-purpose, concurrent, strongly typed, object-oriented** programming language
- Java can be embedded in web pages
- Write once, run anywhere (platform independent)
 - compiled Java code can run on many different operating systems.
- Java LTS versions like **JDK 21** and **JDK 25** are available under Oracle's No-Fee Terms and Conditions (NFTC) license, which allows free use for most purposes, including production, for users and commercial use.
 - **26 (latest release)**
- 2006, Sun announced OpenJDK as FOSS (Free and Open-Source Software) under the terms of the GNU General Public License (GPL). The OpenJDK project itself launched publicly in 2007.
- Java is a *de facto* standard, controlled through the Java Community Process.

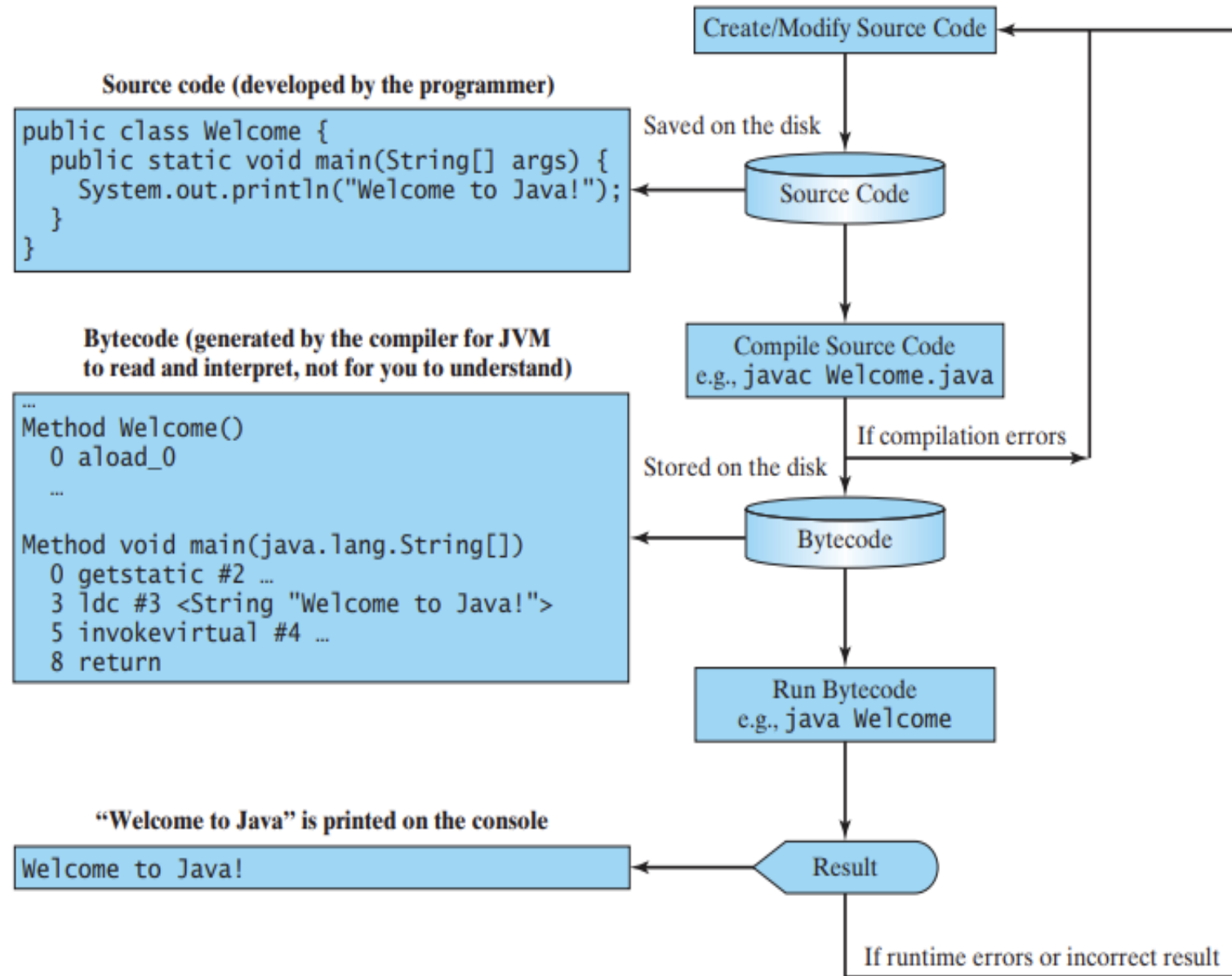


Java Compilation and Execution



- A Java program isn't a single executable file, but instead, it is composed of many individual .class files, each of which corresponds to a single Java class.
- **Java programs are portable.** They can be run on any platform without being recompiled.
- Compilers of most of the programming languages, translate the source code from a high-level programming language directly to a machine code of a given platform, to create an executable program
 - Because different platforms have different machine codes, compilers are different also.
- The Java compiler compiles the source code into the platform-independent bytecode, which is the instruction set of an abstract computing machine, called the **Java Virtual Machine - JVM**.
 - Java bytecode is an intermediate code, because it is not understandable and executable by the real computing device
 - **JVM is platform-dependent**

Java Compilation and Execution



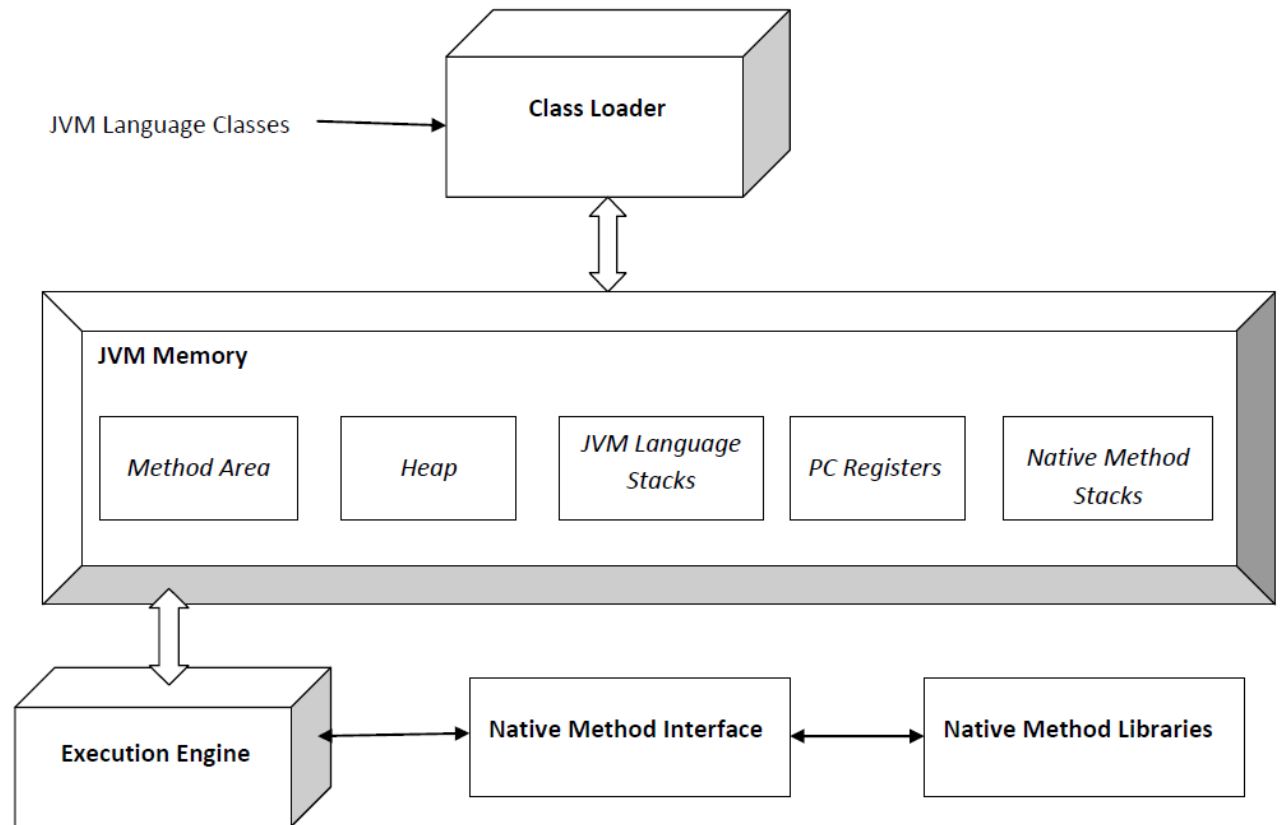
Java Compilation and Execution

There are 3 options for executing bytecodes:

1. In traditional Java execution, JVM interprets bytecode to machine instructions at runtime
 - Big drawback – a Java program is not as fast as running pre-compiled machine instructions (as in C++)
2. Just-in-time (JIT) Compilation - Just before executing the intermediate code, compiling it into the native code, and letting the CPU run the very freshly generated native code. So the compilation is Just In Time of the execution.
 - Just-in-time (JIT) Compilation in JRockit/HotSpot JVMs
3. Ahead-Of-Time (AOT) Compilation - Before everything, before launching the virtual machine, compiling the intermediate code to the native code and letting the CPU run it from the beginning. So the compilation is before/Ahead Of execution Time.
 - Supported in JDK 9

Java Virtual Machine (JVM)

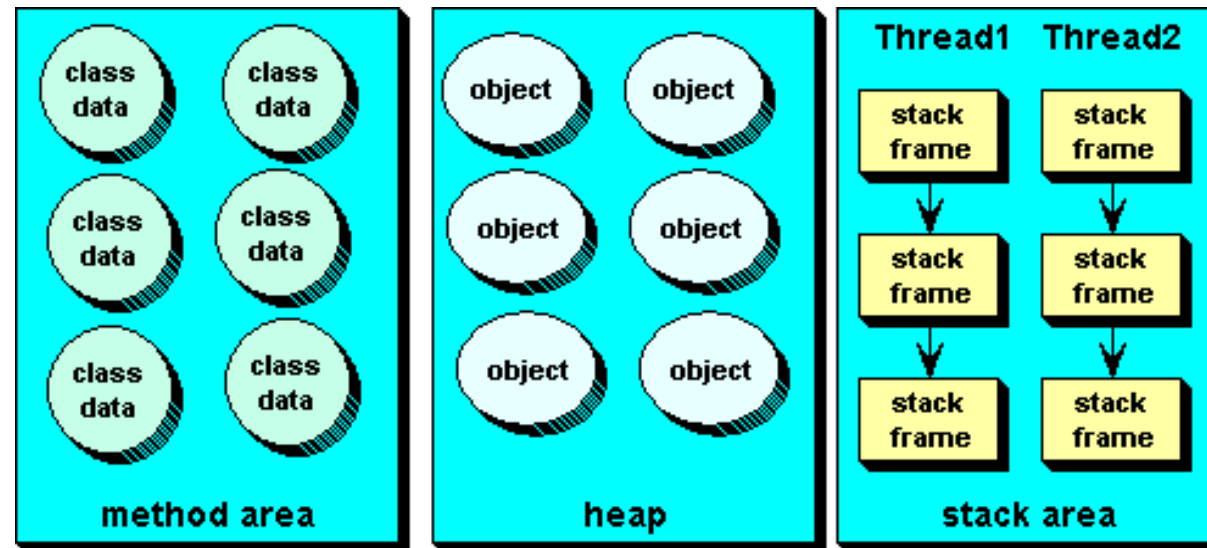
- JVM is a component of the Java system that interprets and executes the instructions in the class files.
- The figure shows a block diagram of the JVM that includes its major subsystems and memory areas



Michelle Ridomi work

Java Virtual Machine (JVM)

- Each instance of the JVM has one **Method area**, one **Heap**, and one or more **Stacks** - one for each thread

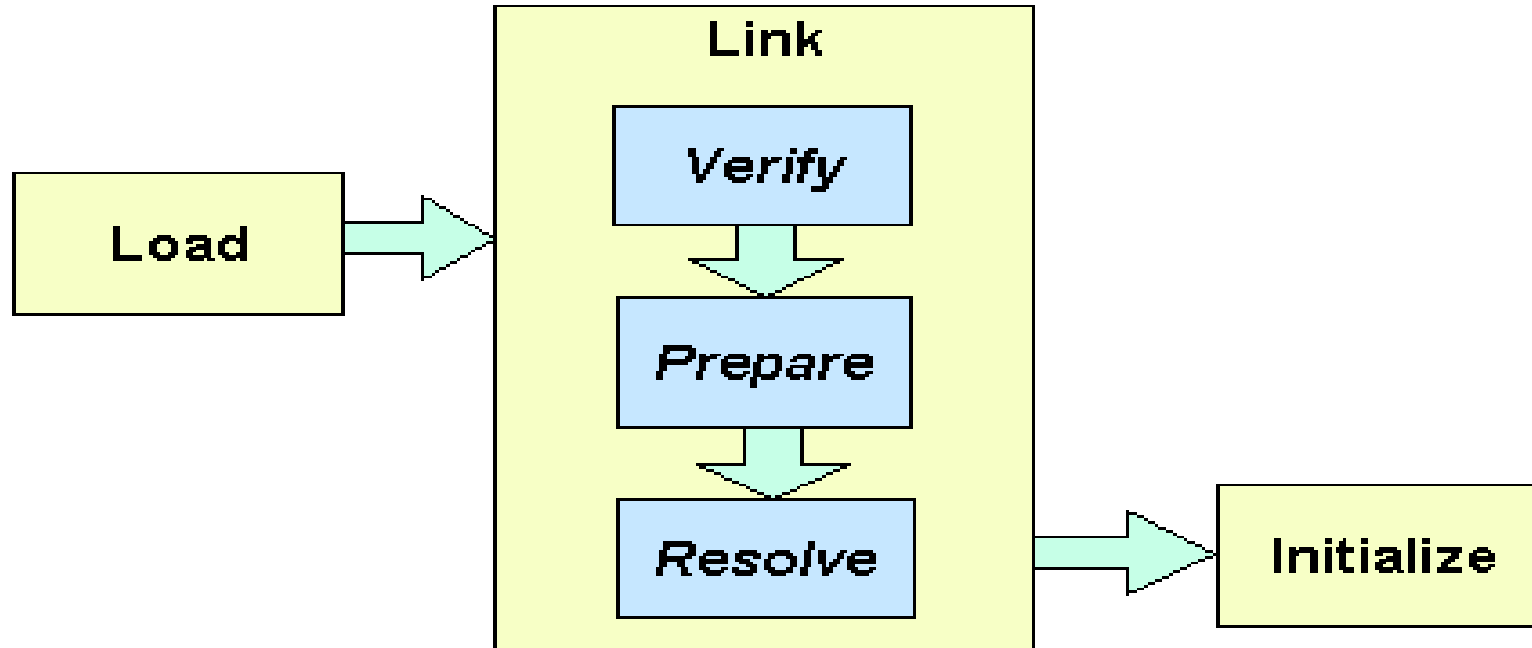


- When JVM loads a class file, it puts its information in the Method area
- As the program runs, all objects instantiated are stored in the Heap
- The Stack area is used to store activation records as a program runs

Java Virtual Machine (JVM)

The Class Loader Subsystem

- The class loader performs three main functions of JVM, namely: **loading**, **linking** and **initialization**.
- The **linking** process consists of three sub-tasks, namely, **verification**, **preparation**, and **resolution**



Java Virtual Machine (JVM)

The Class Loader Subsystem

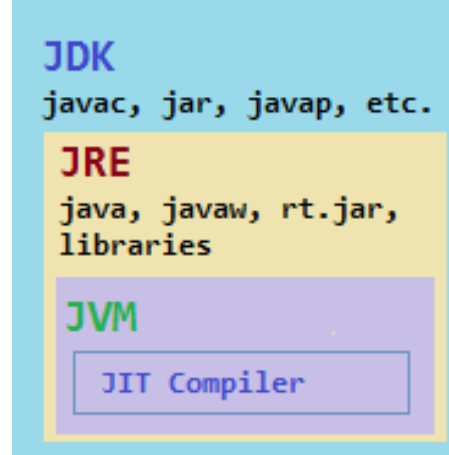
- **Loading** means reading the class file for a type, parsing it to get its information, and storing the information in the Method area.
 - The end of the loading process is the creation of an instance of `java.lang.Class` for the loaded type.
- **Verification** is the process of ensuring that binary representation of a class is structurally correct. Example of some of the things that are checked at verification are:
 - Every method is provided with a structurally correct signature
 - Every instruction obeys the type discipline of the Java language
 - Every branch instruction branches to the start not middle of another instruction
- During the **preparation**, the JVM allocates memory for the static class variables and sets them to default initial values.
- **Resolution** is the process of replacing symbolic names for types, fields and methods used by a loaded type with their actual references.
- **Initialization** is the process of setting class variables to their proper initial values - initial values desired by the programmer.

Java implementations and distributions

Java implementations for Microsoft Windows, Mac OS X, Linux and Solaris.

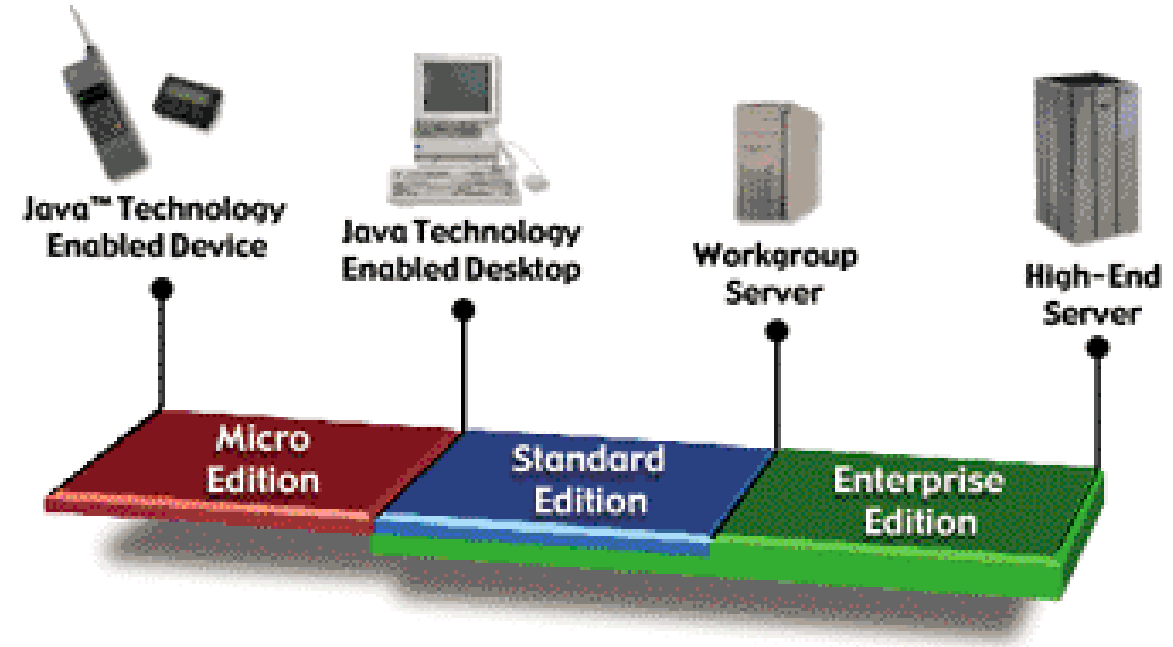
Two different distributions:

- **Java Runtime Environment (JRE)** – a software package that contains what is required to run a Java program, which is a stand-alone Java VM ([HotSpot](#)), browser plugin, Java standard libraries and a configuration tool
 - End users commonly use a JRE installed on their own machine for standalone Java applications, or in a web browser for Java applets.
- **Java Development Kit (JDK)** – a superset of a JRE, that contains tools for Java developers, as:
 - javac – Java compiler
 - javadoc - the documentation generator, which automatically generates documentation from source code comments
 - jar - the archiver, which packages related class libraries into a single JAR file. This tool also helps manage JAR files
 - jdb – the debugger, ...



Java Programming Language Platforms

- Java SE – Java Platform, Standard Edition
- Jakarta EE – Jakarta Platform, Enterprise Edition
- Java ME – Java Platform, Micro Edition



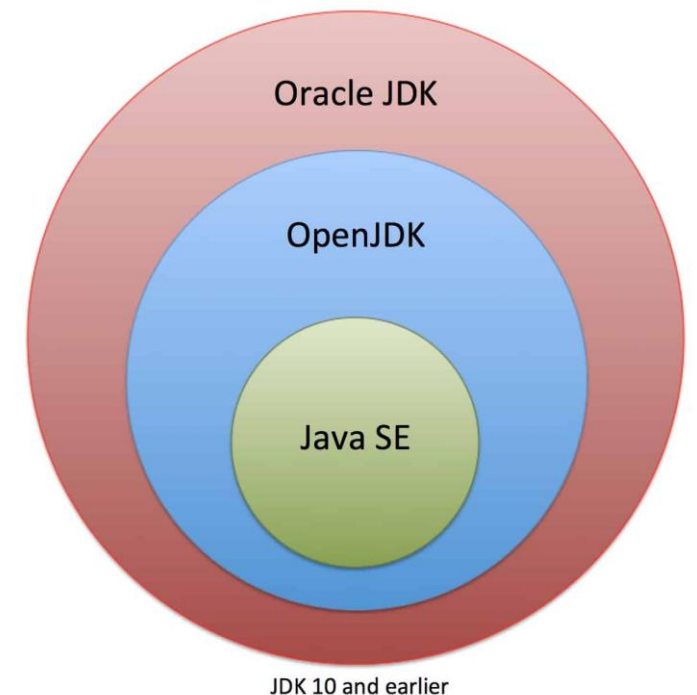
There is also **Java Card** - a specialized Java platform for **smart cards and secure embedded devices**, such as SIM cards, banking cards, and secure elements. It is generally treated separately from the three major Java platform editions.

More Java implementations and IDEs

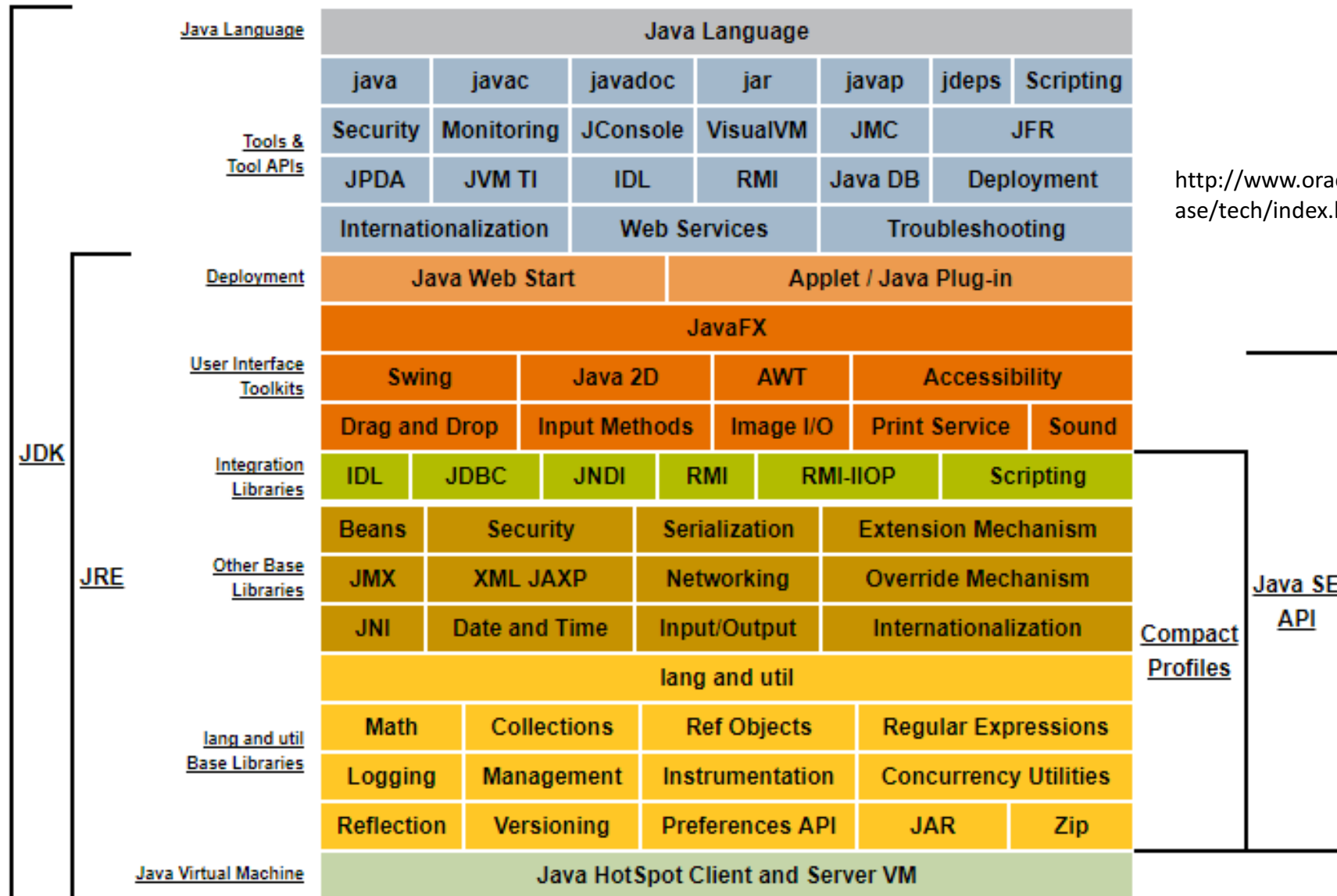
- **OpenJDK** is another Java SE implementation, which is licensed under the GNU GPL, which is the official reference implementation of Java SE since version 7
- **OracleJDK**

Integrated Development Environment (IDE):

- Eclipse, NetBeans, Microsoft Visual Studio, JSource, JBuilder, IntelliJ IDEA (Android Studio), etc.



The Java Platform, Standard Edition (Java SE) 8



Jakarta EE 11 Platform



<https://www.infoq.com/articles/jakarta-ee-11-overview/>

Some Networking Technologies in Java

java.net package

- low-level API, which deals with the following abstractions:
 - *Addresses*, which are networking identifiers, like IP addresses.
 - *Sockets*, which are basic bidirectional data communication mechanisms. TCP and UDP sockets, multicast, secure sockets, etc.
 - *Interfaces*, which describe network interfaces.
- high-level API, which deals with the following abstractions:
 - *URI*, which represent Universal Resource Identifier.
 - *URL*, which represent Universal Resource Locator.
 - *URLConnection*, which is the communication link used to access the resource pointed by the *URL*.
 - *HttpURLConnection* is a subclass of *URLConnection* and provides some additional functionalities specific to the HTTP protocol. This API has been superseded by the newer *HTTP Client API* (11).

java.rmi

- Java Remote Method Invocation
 - A native Java ORB

Deprecated from Java SE 9 till 11

- java.CORBA
- Applet API